

C 语言内存管理编程题

共 3 题 · 难度逐级递增 · 建议用时 150 分钟 · 需上机实现并通过内存检测

题一（入门）：堆上字符串复制 + 动态字符串数组

任务一：实现两个函数

禁止调用 `strdup` / `strndup`，建议自己写循环。

```
#include <stddef.h>

/* 复制整个字符串到新分配的堆内存；失败返回 NULL；s == NULL 返回 NULL */
char *my_strdup(const char *s);

/* 最多复制 n 个字符并补 '\0'；失败返回 NULL；s == NULL 返回 NULL */
char *my_strndup(const char *s, size_t n);
```

任务二：写一个完整程序

- 读入整数 `N` ($0 \leq N \leq 1000$)；
- 接下来 `N` 行，每行长度不超过 1000，可能含空格，也可能为空行；
- 用 `malloc` 建立一个 `char **` 数组保存这 `N` 行（每行都要单独堆分配）；
- 逆序输出所有行；
- 程序结束前释放全部堆内存。

输入样例

```
3
hello world
C memory
ok
```

输出样例

```
ok
C memory
hello world
```

要求

- 任意一次 `malloc` 失败：释放已分配的内存，打印 `alloc failed`，并以非 0 状态码退出。
- `N = 0` 时输出为空且不崩溃。
- 用 `gcc -fsanitize=address,undefined` 或 `Valgrind` 跑样例，必须零泄漏、零越界。

题二（中等）：实现可增长的动态数组 IntVec

任务一：实现下列接口

只能存 `int`，不得把 `realloc` 直接暴露给调用者。

```
#include <stddef.h>

typedef struct {
    int *data;
    size_t size; /* 已用元素个数 */
    size_t capacity; /* 已分配可容纳的元素个数 */
} IntVec;

/* 成功返回 0；失败返回 -1，此时 data == NULL, size == capacity == 0 */
int vec_init(IntVec *v);

/* 保证至少能再放 need 个元素；成功 0，失败 -1；
   失败时 v 的内容和容量必须保持调用前的状态 */
int vec_reserve(IntVec *v, size_t need);

/* 追加一个元素；成功 0，失败 -1；失败时 v 不变，且不得泄漏 */
int vec_push(IntVec *v, int x);

/* 元素个数清零，容量保留 */
void vec_clear(IntVec *v);

/* 释放内存并置为初始状态，允许重复调用 */
void vec_free(IntVec *v);
```

必须满足

- 容量增长使用倍增策略 (1, 2, 4, 8, ...)，保证 `vec_push` 的均摊 $O(1)$ 。
- `vec_reserve` 中若 `need > SIZE_MAX / sizeof(int)`，必须返回 -1，不能发生整数溢出后再分配。
- `realloc` 的返回值必须先用临时指针接住，再赋回 `v->data`（否则失败即泄漏）。

任务二：主程序

读入 `N` ($0 \leq N \leq 10^6$)，再读 `N` 个整数，依次 `vec_push`，用 `qsort` 升序排序后以空格分隔输出，最后 `vec_free`。

输入样例

```
5
3 -1 4 4 0
```

输出样例

```
-1 0 3 4 4
```

加分项：实现 `int vec_shrink_to_fit(IntVec *v)`，把容量缩到恰好等于 `size`。

题三（困难）：在静态内存区上实现 my_malloc / my_free

任务

在一个固定大小的静态字节数组（“模拟堆”）上实现一个**首个适配（first-fit）**分配器：

```
#include <stddef.h>

#define ARENA_SIZE (1u << 20) /* 1 MiB */

void my_init(void); /* 初始化内存区，只能调用一次 */
void *my_malloc(size_t size); /* 成功返回 8 字节对齐的可用内存；失败返回 NULL */
void my_free(void *ptr); /* ptr == NULL 时空操作；否则归还并尽量合并相邻空闲块 */
size_t my_usable_size(void *ptr); /* 该块实际可用的字节数 */
size_t my_free_block_count(void); /* 当前空闲块个数（用于衡量碎片） */
```

必须满足

- 用块头（block header）记录每块的**大小和空闲标志**；块头自身的布局必须保证返回给用户的指针 **8 字节对齐**。
- my_malloc：
 - 从头部开始**首个适配**查找空闲块；
 - 若剩余空间还够放下一个“块头 + 8 字节”，则**分裂**该块；
 - size 过大（含 size + 块头 溢出的情况）必须返回 NULL，绝不能越界；
 - my_malloc(0) 的行为自己定义并写进注释。
- my_free：标记为空闲后，必须与**前、后相邻**的空闲块**合并（coalesce）**，否则多次申请释放后内存会碎成不可用。
- 只允许使用这一个静态数组，**不得再调用 malloc / free**；不要求检测重复释放，但空闲块遍历必须保持一致、不死循环。

任务二：编写测试驱动

- 随机执行 10^5 次 my_malloc / my_free（大小 1..4096），记录所有活跃指针；
- 每个块的用户区**首尾各写一个 canary 字节**，在 my_free 前校验其未被踩坏；
- 断言任意两个活跃块**互不重叠**，且都落在 ARENA_SIZE 范围内；
- 全部释放后，内存应重新拼回一个空闲块（my_free_block_count() == 1）。

评分建议：正确性 40%、合并与碎片控制 25%、边界与失败路径 20%、测试驱动 15%。

提交与检查清单

- 以 gcc -std=c11 -Wall -Wextra -Werror -fsanitize=address,undefined 编译通过、无警告。
- 三题各自的样例输入输出完全一致。
- 边界：N = 0、空行、超大输入、分配失败注入（可用 LD_PRELOAD 或包装宏模拟 malloc 返回 NULL）。

- 所有堆内存存在程序退出前释放完毕，检测工具报告零泄漏、零越界。